What is a GPU and why do we need it in AI?

A simple, in-depth guide for freshers

A GPU is a chip that does thousands of small math calculations at the same time. AI models are built from billions of such calculations, so a GPU runs them far faster than a normal computer chip.

CPU GPU AI



What we will cover

01

CPU basics and GPU basics

02

Why it's called a "Graphics" chip, and its history

03

The math inside an AI model, with a worked example

04

How a CPU vs a GPU does that work

05

Training vs inference, and why training needs GPUs

06

GPU memory and how much a model needs

07

Code you can run yourself

08

Real case studies: AlexNet, Stanford, GPT-3, Llama, ChatGPT

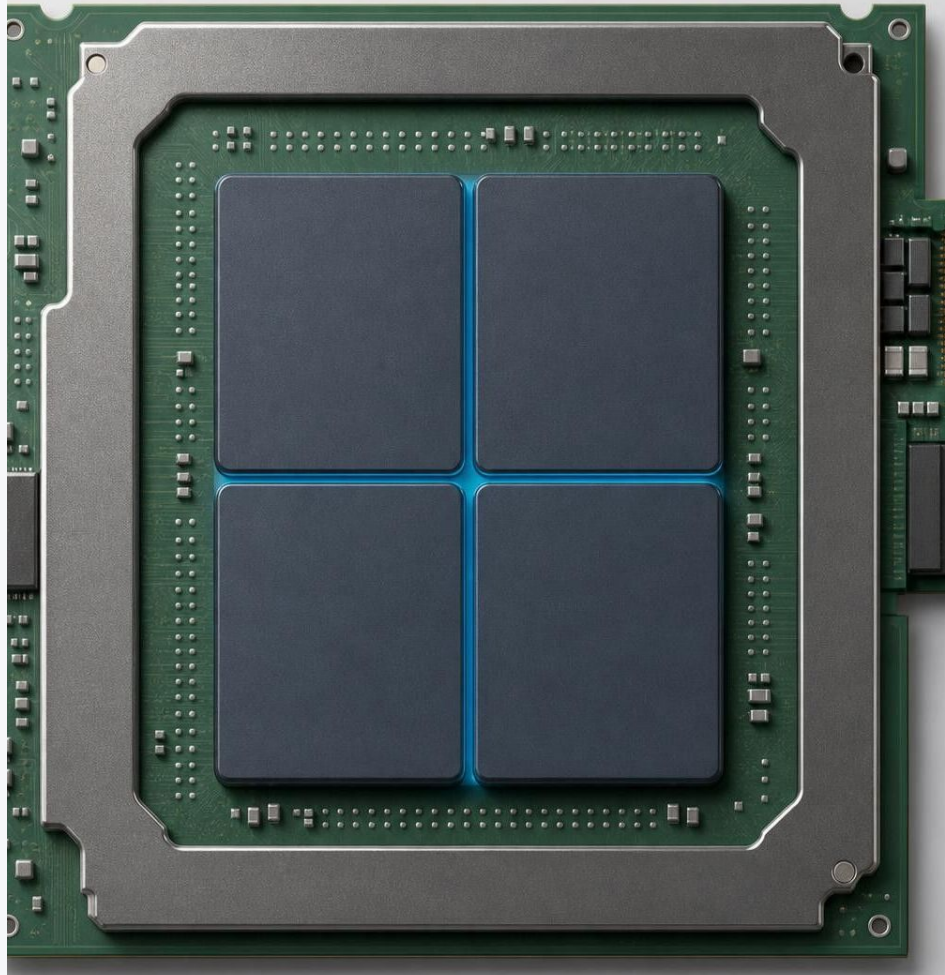
09

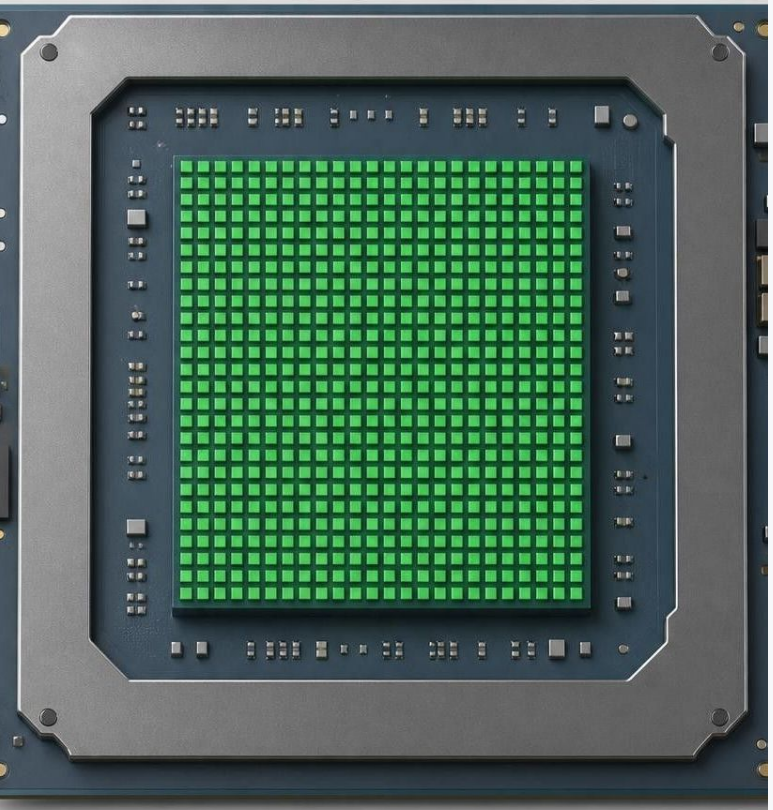
When you don't need a GPU

The CPU: a few powerful cores doing complex work one step at a time

- The CPU (Central Processing Unit) is the computer's main chip. It runs the operating system, the browser, Excel, everything.
- Inside are a few workers called cores; a typical laptop CPU has around 8 to 16.
- Each core is fast and smart, great at decisions like "if this happens, do that, otherwise do something else."
- It is built for low delay on one task, then jumps to the next.

Analogy: a few expert chefs, each cooking one full complex dish at a time.





The GPU: thousands of simple cores working at once

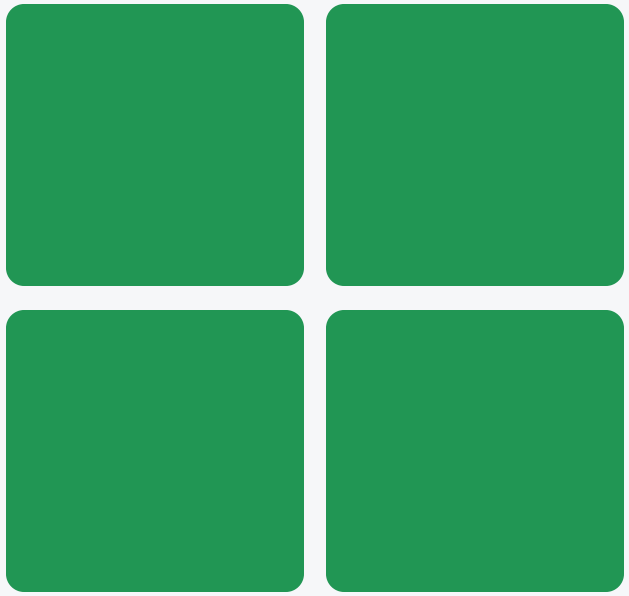
The GPU (Graphics Processing Unit) also has cores, but is built the opposite way. Each core is weaker than a CPU core, but thousands work at the same time. An NVIDIA RTX 4090 gaming card has **16,384** CUDA cores; a data-centre H100 has **16,896**.

CPU	GPU
A few powerful cores	Thousands of small cores
Handles complex logic	Does simple math very fast
One task after another	Many tasks at the same time

Analogy: a huge kitchen of helpers, each chopping one onion at the same moment.

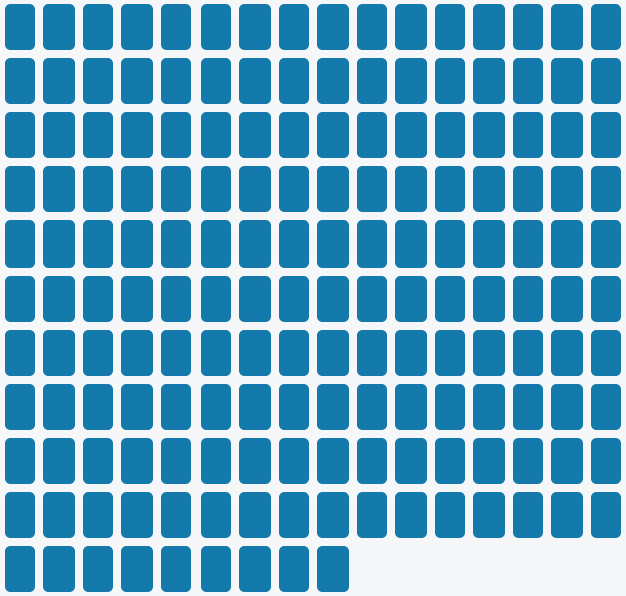
Looking inside: few big cores vs many small cores = parallel processing

CPU



A few big cores

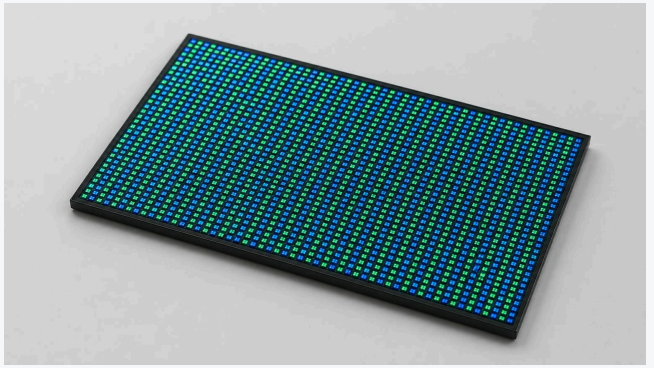
GPU



Many small cores... thousands more in rows

- CPU: a few big cores, one task after another.
- GPU: rows and rows of tiny cores, all working at once.
- Doing many calculations at the same time is called **parallel processing**.
- This is the one thing a GPU is best at, and it's exactly what AI needs.

Why ”Graphics”? GPUs were born to paint pixels for games



01

Screen pixels

A screen is made of millions of tiny dots called pixels. In a game, every pixel's colour is recalculated many times every second.

02

Independent math

One pixel's math does not depend on the next pixel, so all of them can be done at the same time.

03

GPU cores

Chip makers built a chip with thousands of small cores, each calculating a few pixels: that chip is the GPU.

A short history: from gaming chip to AI engine

1999 · GeForce 256

NVIDIA launched it and marketed it as "the world's first graphics processing unit (GPU)".

2006–2007 · CUDA

NVIDIA introduced a way to write normal programs (in C) that run on the GPU, not just graphics.

2012 · AlexNet

Showed GPUs could train deep neural networks far better than before.

Today · GPU clusters

AI companies train models on clusters of tens of thousands of GPUs.



AI math has the same shape as pixel math

Researchers noticed the math inside AI models looks like the math for pixels: lots of small calculations, and each one does not wait for the others. So the chip built for games became the main chip for AI.

1

Pixel math

Many small, independent calculations

2

Same shape

The same shape of work on both sides

3

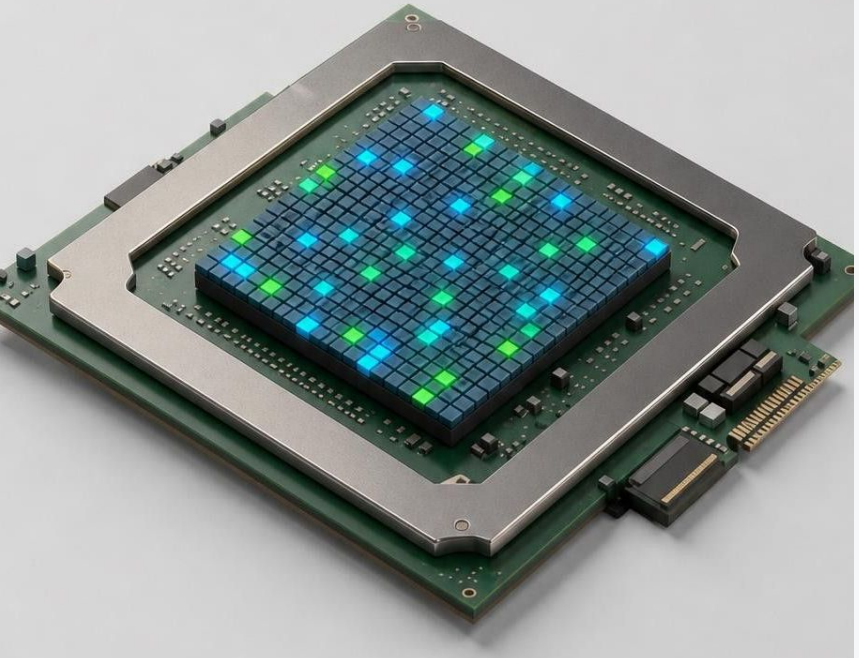
AI math

Many small, independent calculations

4

GPU fits both

One chip design serves games and AI perfectly



Inside an AI model: multiply and add, billions of times

Weights · A model holds billions of numbers called **weights**, which store what it has learned.

Input · Our input (text, image, sound) is turned into numbers too.

Core step · Multiply input numbers by the weights, then add up the results.

This is **matrix multiplication**; a matrix is just a table of numbers in rows and columns. An LLM (large language model) repeats this step **billions of times to produce one word.**

A tiny example: one output number

Made-up numbers, just for understanding.

Input	[2, 3, 4]
Weights	[0.5, 1.0, 0.2]

$$(2 \times 0.5) + (3 \times 1.0) + (4 \times 0.2) = 1.0 + 3.0 + 0.8 = 4.8$$

One output took 3 multiplications and 2 additions. A real model does this with thousands of inputs and billions of weights.

ONE OUTPUT

4.8

3 multiplications · 2 additions

A real model needs thousands of outputs, all independent

INPUT

↓ × weights for output 1

Output 1

...

INPUT

↓ × weights for output 4096

Output 4096

Each layer computes thousands of outputs, and a model has many layers. Each output can be calculated on its own, so all can run at the same time: perfect GPU work.

KEY POINT

Output 2 does not need Output 1 to finish first.

On a CPU: 8 cores grind through about 512 rounds each

Core 1 Output 1 → Output 9 → Output 17 → …

Core 2 Output 2 → Output 10 → Output 18 → …

⋮ cores 3 to 7

Core 8 Output 8 → Output 16 → Output 24 → …

Task: 4,096 outputs on a CPU with 8 cores. Each core finishes one output, then picks the next.

~512

4,096 ÷ 8 = about 512 outputs per core, one after another: many rounds.

4,096

outputs in roughly one round

Thousands of cores, and each takes one output.

Core 1	→	**Output 1**
	⋯	⋯
Core 4096	→	**Output 4096**

All finished in roughly one round, vs ~512 rounds on the CPU.

VERY IMPORTANT

The GPU is not faster because each core is smarter.

It is faster because many cores work on the problem at the same time.

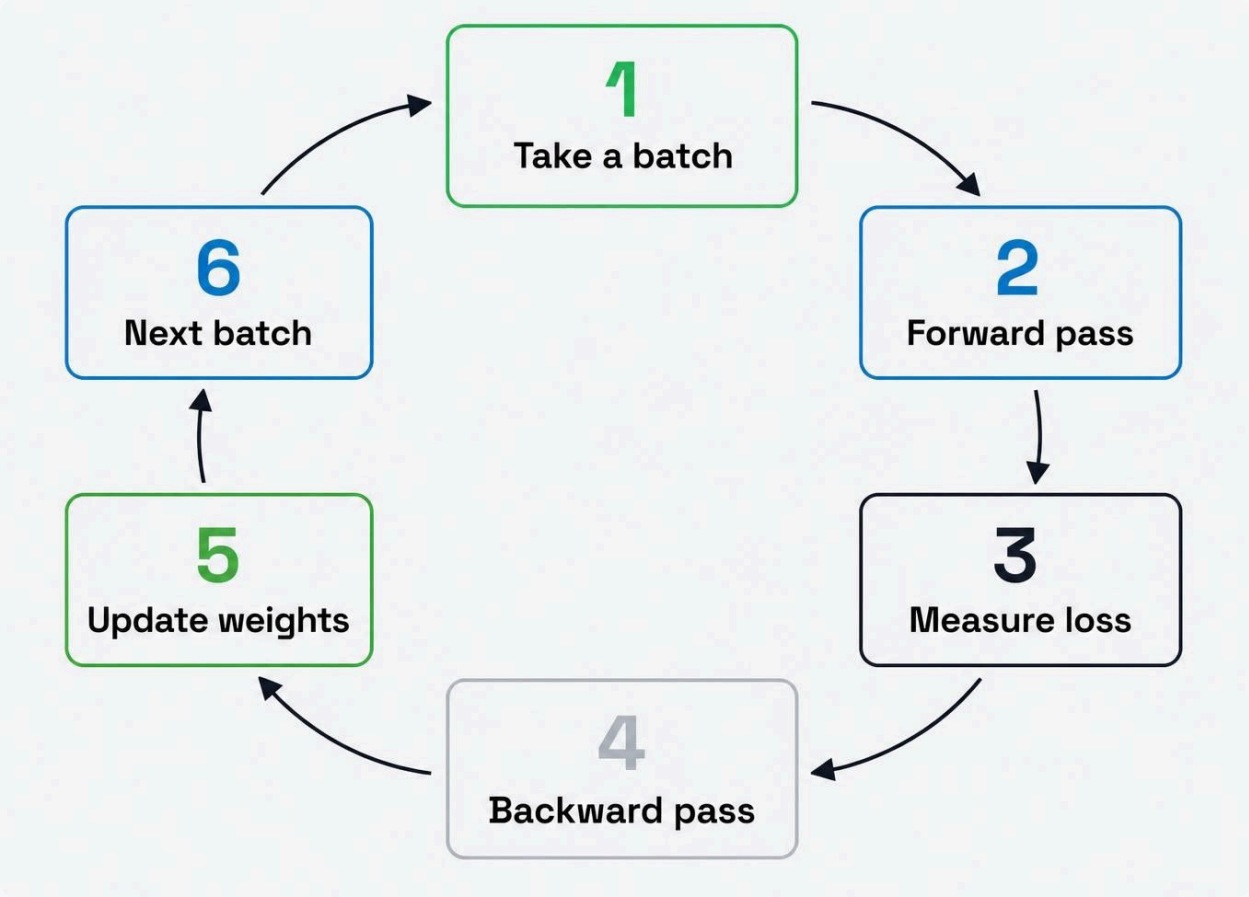
Training teaches the model; inference puts it to work

Training is much heavier than inference. Both use GPUs, but training needs them most.

Training	Inference
The model learns from data	The trained model is used to answer
Weights are adjusted again and again	Weights stay fixed
Very heavy: weeks on thousands of GPUs	Lighter, but needs speed for many users

Example: when you chat with ChatGPT, that's inference; building GPT in the first place was training.

The training loop repeats millions of times



1. Take a batch of data
2. Forward pass: multiply input by weights to make a guess
3. Measure how wrong the guess was (loss)
4. Backward pass: calculate how to change every weight
5. Update all the weights
6. Repeat with the next batch, millions of times

Both passes are huge matrix multiplications, repeated millions of times over billions of weights.

Three reasons training needs GPUs so badly

01

Huge models

Billions of weights, and each one is part of the math.

02

Huge data

LLMs train on trillions of tokens (small pieces of text), each going through the full math.

03

Many repeats

One pass over the data is not enough to learn, so the loop runs again and again.

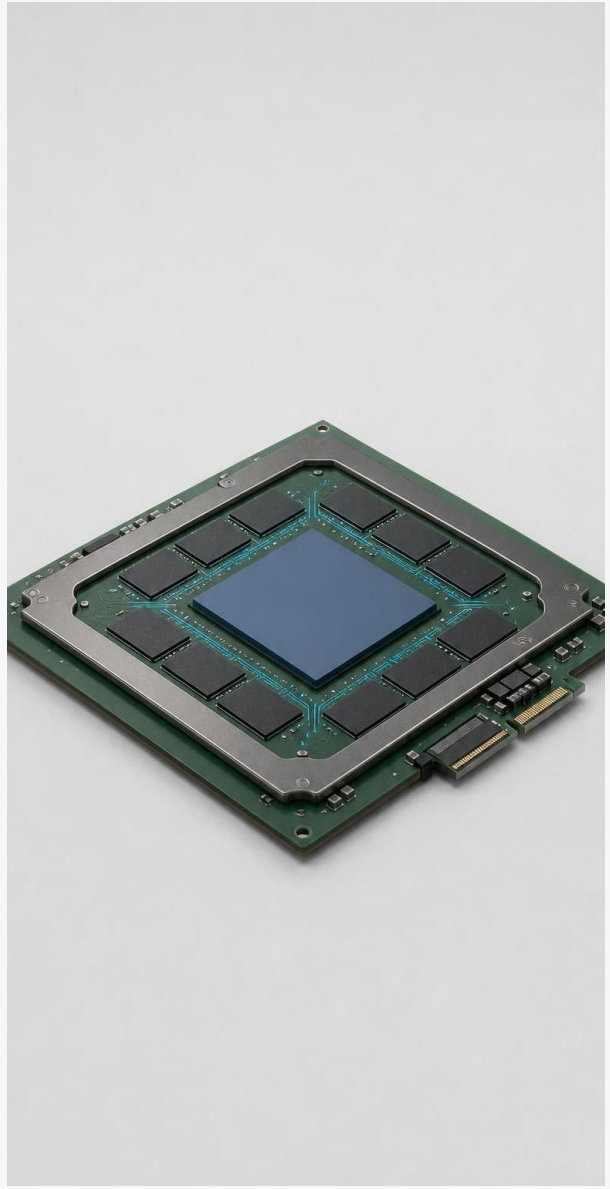
Training that takes weeks on GPUs would take many years on CPUs, which is why large models train on thousands of GPUs together.

GPU memory (VRAM): often the first limit we hit

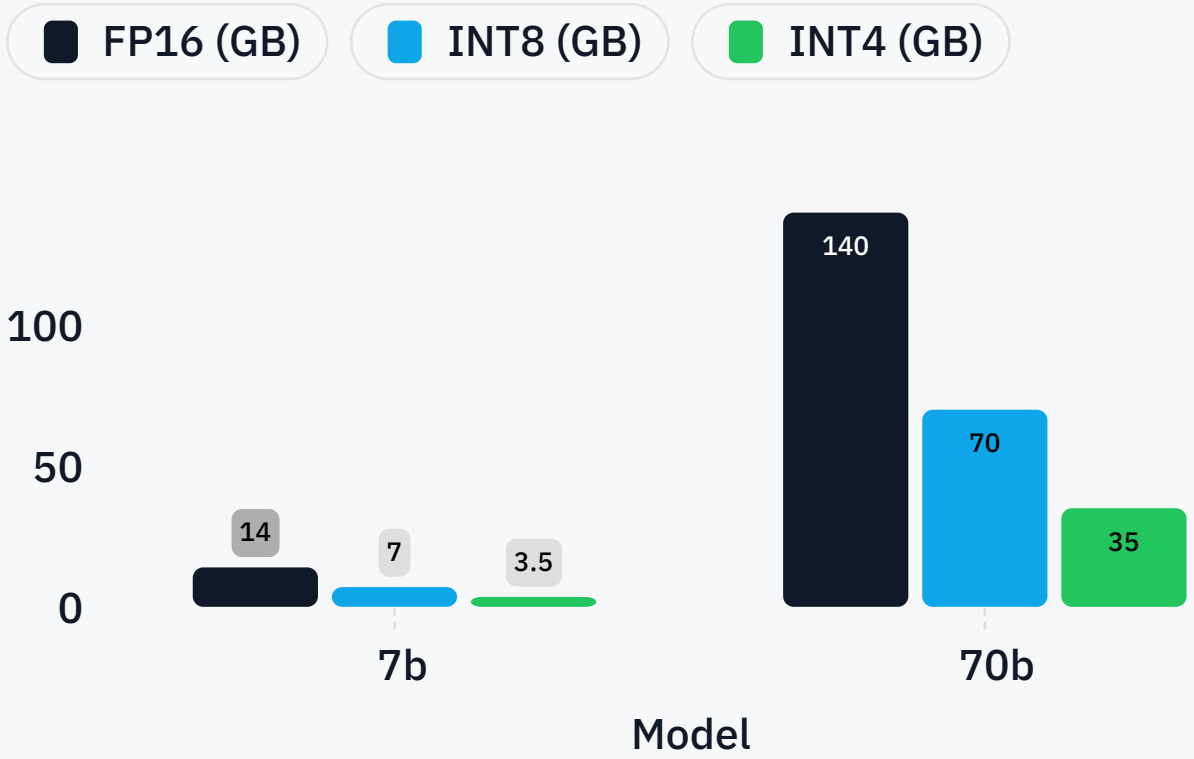
A GPU has its own fast memory, VRAM (Video RAM). Weights and data must be loaded there first. An RTX 4090 has **24 GB**; an H100 has **80 GB** at 3.35 TB/s.

1	2	3	4
Weights on disk	Loaded into VRAM	Cores read and do the math	Output

Too big to fit? You get "out of memory", or split the model across GPUs, or shrink it with **quantization** (fewer bits per weight).



How much memory does a model need? Simple math



7B model: ~14 GB in FP16, ~7 GB in INT8, ~3.5 GB in INT4.

4x

Quantization shrinks memory from FP16 to INT4.

Memory = number of weights × bytes per weight.

FP16 = 2 bytes, INT8 = 1 byte, INT4 = half a byte.

A 70B model needs ~140 GB in FP16—more than one 80 GB H100—but ~35 GB in INT4.

Extra memory is also needed on top while running (the KV cache).

A simple example in code: CPU vs GPU

Same matrix multiplication on CPU, then GPU. Runs on a free Google Colab GPU.

```
import torch, time
size = 4096
a_cpu = torch.rand(size, size)       # random table on CPU
b_cpu = torch.rand(size, size)
start = time.time()
result_cpu = a_cpu @ b_cpu           # @ = matrix multiplication
cpu_time = time.time() - start
a_gpu = a_cpu.to("cuda")             # copy tables into GPU memory
b_gpu = b_cpu.to("cuda")
_ = a_gpu @ b_gpu                    # warm-up run
torch.cuda.synchronize()
start = time.time()
result_gpu = a_gpu @ b_gpu
torch.cuda.synchronize()             # wait for the GPU to finish
gpu_time = time.time() - start
print(f"CPU: {cpu_time:.3f}s | GPU: {gpu_time:.3f}s")
print(f"GPU was about {cpu_time / gpu_time:.0f}x faster")
```

What the code shows: same work, different place

Same work both times: multiplying two tables of about **16.7 million** numbers each. Only where it runs changes; on a typical Colab setup the GPU finishes many times faster. And this is one multiplication; a model does millions.

`.to("cuda")`

Copies data into GPU memory; the GPU only works on data in its own memory.

`synchronize()`

`torch.cuda.synchronize()` makes Python wait until the GPU really finishes, or the timer stops too early.

Warm-up run

Runs the math once first, so one-time setup cost isn't counted in the timing.

AlexNet (2012): two gaming GPUs that changed AI

15.3%

AlexNet · top-5 error

26.2%

Runner-up · top-5 error

AlexNet won the 2012 ImageNet challenge.

- University of Toronto researchers trained this image-recognition network on two NVIDIA GTX 580 gaming GPUs, with 3 GB each.
- Training took about 5–6 days. The network was split into two halves, one per GPU, because one card's memory wasn't enough.

Lesson: GPUs made deep learning practical, and memory limits shaped the design.





3 GPU servers matched 1,000 CPU servers

3

GPU servers · Stanford, 2013

1,000

Google CPUs · 16,000 cores

- Google's CPU network learned to recognise cats in YouTube videos; Andrew Ng's Stanford team built an equally large one with just three GPU servers.
- Their system trained 1-billion-weight networks on 3 machines in a couple of days, and scaled to 11 billion weights on 16 machines.

Lesson: parallel GPU math gives the same result with far less hardware and cost.



GPT-3, Llama and ChatGPT run on GPU armies

24,576 NVIDIA H100 GPUs

per Llama 3 training cluster

- **GPT-3:** Microsoft built OpenAI a supercomputer with 10,000 GPUs and 285,000 CPU cores.
- **Llama 2:** Meta's pretraining used about 3.3 million A100 GPU-hours; the 70B model alone took about 1.7 million.
- **ChatGPT:** Microsoft connected tens of thousands of NVIDIA A100 chips to build the hardware behind it.
- Note the CPUs in GPT-3's machine: CPU and GPU always work as a team.

The GPU doesn’t replace the CPU: they work together

1. CPU reads files and prepares data
2. Sends the heavy math to the GPU
3. GPU does the matrix multiplications
4. GPU sends results back to the CPU

Point	CPU	GPU
Number of cores	Few (around 8 to 64)	Thousands
Each core	Powerful, complex logic	Simple, basic math fast
Best at	One step after another	Many small tasks at once
Memory	Computer's main RAM	Its own fast memory (VRAM)
Role in AI	Runs the program, prepares data	Does the heavy matrix math

Do we always need a GPU? No, and here's how to decide

NO GPU NEEDED

Small models like linear regression or a small decision tree; using an LLM through an API (the provider runs the GPUs); small quantized language models on a laptop CPU, just slower.

GPU NEEDED

Training deep learning models, fine-tuning large models, running large models yourself with fast responses.

No GPU at home? Google Colab and Kaggle give free GPU time in the browser.

Remember: a GPU wins because AI math is made of many small calculations that can all happen at the same time.

